Quiz 5 Solution

1. Which declaration defines a function template named `max` that takes two parameters of the same type `T` by value and returns the larger one?

A. template<typename T> T max(T a, T b) { return a > b ? a : b; }
B. template<typename T> T max(const T* a, const T* b) { return *a > *b ? *a : *b; }
C. template<T> T max(T a, T b) { return a > b ? a : b; }
D. template<typename T> T max(T a) { return a; }

Answer: A

2. Which of these correctly declares a class template named `SimpleList` with exactly one type parameter `T`?

A. template<class T> class SimpleList { /*...*/ };
B. class template<class T> SimpleList { /*...*/ };
C. template<T> class SimpleList { /*...*/ };
D. template<class> class SimpleList<T> { /*...*/ };

Answer: A

3. Given the class template below, which line instantiates an object `jd` for `double` values?

```
template<class T>
class Joiner {
public:
  T combine(T x, T y) { return x + y; }
};
```

A. Joiner<double> jd;
B. Joiner jd<double>;
C. Joiner<double>();
D. Joiner<> jd;

Answer: A

4. Which syntax shows the explicit specialization of the class template `A` for the pointer type `char*`?

A. template<> class A<char*> { /*...*/ };
B. class A<char*> : public A { /*...*/ };
C. template<class char*> class A { /*...*/ };
D. template<char*> class A<> { /*...*/ };

Answer: A

5. Which is the correct syntax to **define** the member function memberName() **outside the class body**, given the following class template declaration?

```
template<typename T>
class ClassName {
public:
  void memberName();
};
```

A. template<typename T>
void ClassName<T>.memberName() { /*...*/ }

B. template<typename T>
void ClassName<T>::memberName() { /*...*/ }

C. template<typename T>
void ClassName::memberName()<T> { /*...*/ }

D. template<typename T>
void ClassName<T>::template memberName() { /*...*/ }

Answer: B

6. What is the correct way to define the member function `combine`, which takes two arguments of type T and returns a value of type T, outside the class body for the template class `Joiner<T>`?

A. template<class T>
T Joiner<T>::combine(T x, T y) { return x + y; }
B. template<T>
T Joiner<T>::combine(T x, T y) { return x + y; }
C. T template<class T>
Joiner<T>::combine(T x, T y) { return x + y; }
D. template<class T>
Joiner::combine(T x, T y) T { return x + y; }

Answer: A

7. Which of these is the explicit specialization of `template<typename T1, typename T2> class A` for `<double, int>`?

A. template<> class A<double, int> { /*...*/ };
B. template<typename T1> class A<T1, int> { /*...*/ };
C. template<typename T1, typename T2> class A { /*...*/ };
D. class A<double, int> { /*...*/ };

Answer: A

8. How can you declare a template class MyClass, such that it uses int as the **default type argument** when no type is provided?

A. template<typename T = int>

class MyClass { /* ... */ };

B. template<int T = int>

class MyClass { /* ... */ };

C. template<typename T>

class MyClass<int> { /* ... */ };

D. template<typename T = int, typename U>

class MyClass { /* ... */ };

Answer: A

9. Which declaration will use the **partial specialization** given the following general class template:

template<typename T1, typename T2>

class A { /* ... */ };

And a **partial specialization**:

template<typename T2>

class A<double, T2> { /* ... */ };

A. A<int, float> a;
B. A<double, float> a;
C. A<int, double> a;
D. A<char, double> a;

Answer: B

10. Which declaration is the unspecialized (primary) template for a class `A` with two type parameters?

A. template<typename T1, typename T2> class A { /*...*/ };
B. template<typename T1> class A<T1, int> { /*...*/ };
C. template<> class A<double, int> { /*...*/ };
D. class A { /*...*/ };

Answer: A

11. Which of the following statements about function templates is INCORRECT?

A. The same function template can be used with different data types

B. Each type used with a function template results in a separate version of the function

C. A function template must be rewritten for every new data type

D. A function template can have more than one template parameter

Answer: C

12. What will be the output of this code?

```
int main(){

    template <typename T>

    T a = 10;
    cout<<a;

    a = "Quiz!";
    cout<<a;

    return 0;
}
```

A. 10 Quiz!

B. Quiz! 10

C. Compile-time error

D. None of the others

Answer: C

13. What will be the output of this code?

```
template <typename T>
class Calc{

    public:
    T add(T a,T b){  return a + b; }
};

int main()
{
    Calc obj;
    cout<<obj.add(3,4);

    return 0;
}
```

A. 7

B. 7.0

C. Compile-time error

D. None of the others

Answer C

14. Which one of these is the correct way to initialize a template in C++?

A. template typename T

B. template <typename T>

C. template (typename T)

D. <template typename T>

Answer B

15. Which of these statements is CORRECT about specialized templates?

A. They override the behavior of general templates

B. They have the same precedence as general templates

C. They are evaluated at runtime

D. None of the others

Answer: A

16. What is the purpose of template specialization in C++?

A. To create a more efficient main function

B. To allow custom behavior for specific template arguments

C. To define default values for parameters

D. To prevent templates from being instantiated

Answer: B

17. What is the correct way to declare a template function that adds two values?

A. template <add(T a, T b)>

B. template <typename T> T add(T a, T b)

C. function<T> add(T a, T b)

D. template (T) add(a, b)

Answer: B